```
module {
  deepgen.kernel @attention(%Q : memref<1x32x4096x128xf32>, %K : memref<...>, %V : memref<...>, %O : memref<...>) {
    deepgen.parallel %bz, %by, %bx = (1, 32, 128), thread_num = 256 {
      %smemQ = deepgen.alloc_buffer {scope = "shared", alignment = 16 : i64} -> memref<128x32xf32>
      ...  // alloc buffers
      deepgen.fill %max {value = 0xFF800000 : f32} : memref<32x32xf32>
      ...  // init buffers
      deepgen.copy %smemQ[0, 0], %Q[%bz, %by, %bx * 32, 0] : memref<128x32xf32>, memref<1x32x4096x128xf32>
      // === mian loop ===
      deepgen.for %i = 0 to 4096 step = 32 {
        // ### Matmul-QK ###
        deepgen.fill %acc_s {value = 0.000000e+00 : f32} : memref<32x32xf32>
        deepgen.for %k = 0 to 128 step 16 {
          deepgen.copy %smemK[0, 0], %K[%bz, %by, %i, %k] : memref<16x32xf32>, memref<1x32x4096x128xf32>
          deepgen.matmul %smemQ[%k, 0], %smemK, %acc_s : memref<128x32xf32>, memref<16x32xf32>, memref<32x32xf32>
        }
        // ### Softmax ###
        deepgen.copy %prev_max[0], %max[0] : memref<32xf32>, memref<32xf32>
        deepgen.fill %max {value = 0xFF800000 : f32} : memref<32x32xf32>
        deepgen.reduce %max, %acc_s {kind = "max", dim = 1, clear = 0} : memref<32xf32>, memref<32x32xf32>
        deepgen.block %j = 0 to 32 {                    // scale = exp(prev_max - max)
          %0 = affine.load %prev_max[%j] : memref<32xf32>
          %1 = affine.load %max[%j] : memref<32xf32>
          %2 = arith.subf %0, %1 : f32
          %3 = math.expf %2 : f32
          affine.store %3, %scale[%j] : memref<32xf32>
        }
        deepgen.block %j, %k = 0, 0 to 32, 32 { ... }   // acc_s = exp(acc_s - max)
        deepgen.reduce %sum, %acc_s {kind = "sum", dim = 1, clear = 1} : memref<32xf32>, memref<32x32xf32>
        deepgen.block %j = 0 to 32 { ... }              // log_sum = log_sum * scale + sum
        deepgen.block %j, %k = 0, 0 to 32, 32 { ... }   // acc_o = acc_o * scale
        // ### Matmul-PV ###
        deepgen.copy %smemV[0, 0], %V[%bz, %by, %i, 0] : memref<32x128xf32>, memref<1x32x4096x128f32>
        deepgen.matmul %acc_s, %smemV, %acc_o : memref<32x32xf32>, memref<32x128xf32>, memref<32x128xf32>
      }
      deepgen.block %j, %k = 0, 0 to 32, 32 { ... }     // acc_o = acc_o / log_sum
      deepgen.copy %O[%bz, %by, %bx * 32, 0], %acc_o[0, 0] : memref<1x32x4096x128f32>, memref<32x128xf32>
    }
  }
}
```